

Rapport du projet Oz

Sharelock OZmes Web3son

Aurelle Donhoungo 95942400

Gerald Crisostomo 18582501

May 2026

1 Introduction

Dans le cadre du cours LINFO1104, nous avons mis en œuvre une version simplifiée d'une blockchain dans Oz. L'objectif étant de traiter une liste de transactions, de construire la blockchain qui en résulte et de décoder la phrase secrète cachée par Sharelock OZmes. Nous avons également mis en œuvre trois extensions à savoir: Denylist, L'effort n'est pas gratuit ainsi que Fêru de stats.

2 Choix d'implémentation

La procédure **ExecuteBlockchain** commence par convertir le bloc de genèse en un état initial à l'aide de **ConvertGenesis** qui utilise Arity et FoldL pour construire un enregistrement où chaque utilisateur est représenté par `user(balance:B nonce:0)`. Elle initialise ensuite un bloc de genèse fictif avec un numéro défini sur -1 qui sert de point de départ à la chaîne. Le traitement est envoyé à **ProcessBlocks** qui est une fonction récursive qui regroupe chaque bloc pour les transactions via **GroupByBlock** ensuite filtre les transactions valides via **FiltreTransaction** en vérifiant simultanément leur validité et l'effort cumulé maximal de 300, construit le bloc via **CreeBlock** en calculant le hachage final puis s'appelle elle-même de manière récursive avec l'état mis à jour.

La validation d'une transaction dans **TransactionOk** vérifie six conditions de manière suivante: le nonce, le hachage, un solde suffisant, une valeur positive, un `max_effort` positif et un effort inférieur à `max_effort`. L'état est un enregistrement Oz dont les caractéristiques sont les identifiants des utilisateurs. Nous utilisons `HasFeature` pour vérifier l'existence d'un utilisateur et `AdjoinAt` pour le mettre à jour. Cette approche est entièrement déclarative sans cellules ni constructions non déclaratives.

3 État final et blockchain

La phrase secrète obtenue est : **prof peter van royiarty is behind oz**

Utilisateur	Solde	Nonce																																								
11	97415	2	<table><tr><th>Bloc</th><th>Hash</th><th>PrevHash</th><th>Tx hashes</th></tr><tr><td>0</td><td>256424</td><td>0</td><td>22472, 38896, 81865, 61514, 51677</td></tr><tr><td>1</td><td>153662</td><td>256424</td><td>363117, 465569, 68551</td></tr><tr><td>2</td><td>146614</td><td>153662</td><td>751304, 50438, 62480, 128728</td></tr><tr><td>3</td><td>647331</td><td>146614</td><td>291119, 170096, 24257, ...</td></tr><tr><td>4</td><td>102336</td><td>647331</td><td>247790, 139871, ...</td></tr><tr><td>5</td><td>272434</td><td>102336</td><td>92750, ...</td></tr><tr><td>6</td><td>551027</td><td>272434</td><td>...</td></tr><tr><td>7</td><td>297173</td><td>551027</td><td>...</td></tr></table>				Bloc	Hash	PrevHash	Tx hashes	0	256424	0	22472, 38896, 81865, 61514, 51677	1	153662	256424	363117, 465569, 68551	2	146614	153662	751304, 50438, 62480, 128728	3	647331	146614	291119, 170096, 24257, ...	4	102336	647331	247790, 139871, ...	5	272434	102336	92750, ...	6	551027	272434	...	7	297173	551027	...
Bloc	Hash	PrevHash					Tx hashes																																			
0	256424	0					22472, 38896, 81865, 61514, 51677																																			
1	153662	256424					363117, 465569, 68551																																			
2	146614	153662					751304, 50438, 62480, 128728																																			
3	647331	146614					291119, 170096, 24257, ...																																			
4	102336	647331					247790, 139871, ...																																			
5	272434	102336					92750, ...																																			
6	551027	272434					...																																			
7	297173	551027					...																																			
14	70294080	8																																								
15	94650918	6																																								
32	5141804	7																																								
37	1239534151	0																																								
49	463119	1																																								
60	36160	3																																								
61	221043	4																																								
66	480590	2																																								
73	58497	4																																								
86	918173	2																																								
87	210757	1																																								
100	579432	3																																								

Table 2: Blockchain finale

Table 2: Blockchain finale

Table 1: État final des utilisateurs

4 Extensions

4.1 Denylist

Un utilisateur est ajouté à la denylist s'il envoie 3 transactions ou plus dans un même bloc, qu'elles soient valides ou non. Une fois blacklisté, toutes ses futures transactions sont ignorées dans les blocs suivants. Les transactions envoyées avant le blacklistage restent valides. La denylist est stockée dans l'état sous le champ denylist, initialisé à **nil** dans **ConvertirGenesis**. Après le traitement de chaque bloc, **UpdateDenylist** parcourt toutes les transactions du bloc et blacklist les utilisateurs ayant envoyé 3 transactions ou plus via CountSenderTxns. La mise à jour de la denylist se fait après le filtrage des transactions du bloc courant, afin que les transactions envoyées avant le blacklistage soient prises en compte si elles sont valides.

- État final avec Denylist
Utilisateurs blacklistés : 32, 15, 73

Utilisateur	Solde	Nonce				
11	97415	2	Bloc	Hash	Prev	Tx hashes
14	70226791	8				
15	94043702	6	0	256424	0	22472, 38896, 81865, 61514, 51677
32	6107292	3	1	153662	256424	363117, 465569, 68551
37	1239534151	0	2	146614	153662	751304, 50438, 62480, 128728
49	463119	1	3	356212	146614	170096, 24257, 15242
60	36160	3	4	743877	356212	247790, 139871
61	221043	4	5	913975	743877	92750, ...
66	457435	2	6	168063	913975	...
73	81652	3	7	914209	168063	...
86	918173	2				
87	210757	1				
100	288449	3				

Table 4: Blockchain avec Denylist

Table 3: État final avec Denylist

Phrase secrète avec Denylist : **prof petezpc h g qh gp ejzjwx xi**

Pour activer cette extension : **make run-denylist**

4.2 Extension Effort payant

Le coût de l'effort est déduit du solde du sender en plus du montant de la transaction. Si le sender ne dispose pas de suffisamment de fonds pour couvrir $\text{value} + \text{effort}$, la transaction est refusée. Deux fonctions ont été modifiées par rapport à BaseModule.oz. Dans TransactionOk, la condition SenderOk vérifie désormais que le solde du sender est supérieur ou égal à $\text{value} + \text{effort}$. Dans ApplyTransaction, le nouveau solde du sender est calculé en déduisant à la fois le montant et l'effort : $\text{NewSenderBalance} = \text{SenderBalance} - \text{value} - \text{effort}$.

- État final avec Effort payant

Phrase secrète avec Effort payant : **prof petezpc h g qh gp ejzjwx xi**

Pour activer cette extension : **make run-effort**

- L'effort n'est pas gratuit

Utilisateur	Solde	Nonce
11	97353	2
14	70226351	8
15	94043452	6
32	6107199	3
37	1239534151	0
49	463088	1
60	36035	3
61	220919	4
66	457341	2
73	81527	3
86	918127	2
87	210726	1
100	288292	3

Table 5: État final avec Effort payant

4.3 Féru de stats

Cette extension ajoute trois fonctions statistiques sur la blockchain finale. La liste de toutes les transactions est extraite via **GetAllTransactions** qui parcourt chaque bloc et concatène ses transactions.

BiggestSenders compte le nombre de transactions envoyées par chaque sender via **CountSenders** qui maintient une liste paires id count. Cette liste est triée par ordre décroissant via un tri par insertion **Sort** qui gère les égalités par ordre croissant d'identifiant. Les N Premiers sont extrait via **FirstN** et retourne -1 si $N \leq 0$.

ReceivedMost suit la même logique mais accumule la valeur totale reçue par chaque receiver via **CountReceivers**.

MoneyPeak rejoue toutes les transactions depuis un état vide en maintenant via **TrackPeaks** le solde maximum jamais atteint par chaque utilisateur. **FindMax** retourne ensuite la paire avec le solde le plus élevé.

- Résultats obtenus:

Fonction	Résultat
BiggestSenders (Top 3)	[14#8, 32#7, 15#6]
ReceivedMost (Top 3)	[100#1066042 86#986359 15#786539]
MoneyPeak	100#1066042

Table 6: Résultats des fonctions statistiques

Pour Activer cette extension: `make run-stats`

5 Conclusion

Ce projet nous a permis de mettre en place une blockchain fonctionnelle dans Oz, capable de traiter des transactions, de créer des blocs et de décoder une phrase secrète. Les trois extensions que nous avons développées Denylist, Effort et Feru stats viennent enrichir le système de base ceci en triant des utilisateurs qui ont des transactions erronées. Ce travail nous a permis de mettre en pratique les concepts fondamentaux d'Oz, notamment la programmation déclarative et la récursivité.